



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Microsoft Certified: Azure Administrator Associate (AZ-104)

TGS Ref No: TGS-2023039182

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	1 July 2026	Initial release — AZ-104 Learner Guide covering all 26 labs.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction.....	4
Course Learning Outcomes.....	4
Before You Start — Environment Setup.....	4
Topic 01 — Manage Azure Identities and Governance (20–25%).....	5
Lab 1 — Manage Microsoft Entra Users and Groups.....	5
Lab 2 — Manage External (B2B Guest) Users and Configure SSPR.....	6
Lab 3 — Manage Access to Azure Resources with RBAC.....	8
Lab 4 — Implement and Manage Azure Policy.....	9
Lab 5 — Manage Resource Groups, Locks and Tags.....	10
Lab 6 — Manage Costs and Configure Management Groups.....	11
Topic 02 — Implement and Manage Storage (15–20%).....	13
Lab 7 — Create and Configure Storage Accounts and Redundancy.....	13
Lab 8 — Configure Access to Storage: Firewalls, SAS, Access Keys.....	14
Lab 9 — Configure Azure Blob Storage: Tiers, Lifecycle, Versioning.....	16
Lab 10 — Configure Azure Files and Identity-Based Access.....	17
Lab 11 — Manage Data with Storage Explorer and AzCopy.....	18
Topic 03 — Deploy and Manage Azure Compute Resources (20–25%).....	19
Lab 12 — Automate Deployment with ARM Templates and Bicep.....	20
Lab 13 — Create a Virtual Machine, Encryption at Host, and Moving VMs.....	21
Lab 14 — VM Sizes, Disks, and Availability (Zones & Sets).....	22
Lab 15 — Deploy and Configure a Virtual Machine Scale Set (VMSS) with Autoscale.....	23
Lab 16 — Containers: ACR, Container Instances, and Container Apps.....	24
Lab 17 — Azure App Service: Plans, Scaling, TLS, Custom Domains, Backup, Networking, Slots.....	26
Topic 04 — Implement and Manage Virtual Networking (15–20%).....	27
Lab 18 — Virtual Networks, Subnets & VNet Peering.....	27
Lab 19 — Public IPs, User-Defined Routes & Connectivity Troubleshooting.....	29
Lab 20 — Network Security Groups, Application Security Groups & Effective Rules.....	30
Lab 21 — Azure Bastion, Service Endpoints & Private Endpoints.....	32
Lab 22 — Azure DNS & Load Balancing.....	33
Topic 05 — Monitor and Maintain Azure Resources (10–15%).....	35
Lab 23 — Azure Monitor: Metrics, Logs, KQL, and Alerts.....	35
Lab 24 — Azure Network Watcher and Connection Monitor.....	36
Lab 25 — Azure Backup: Recovery Services Vault, Policies, Backup and Restore.....	38
Lab 26 — Azure Site Recovery: Replicate, Fail Over, and Interpret Health.....	39
Exam Preparation.....	40
Glossary.....	41

Introduction

This Learner Guide accompanies the WSQ course Microsoft Certified: Azure Administrator Associate (AZ-104) (TGS-2023039182), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 26 hands-on labs, organised by the five official AZ-104 exam skill areas. Every lab maps to a published exam objective and is completed in the Azure Portal together with Azure Cloud Shell (Azure CLI and PowerShell).

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. Each lab creates its own resource group (rg-az104-labNN); always run the clean-up step at the end of a lab to avoid unnecessary Azure charges.

Course Learning Outcomes

- LO1: Manage Azure identities and governance — Microsoft Entra users and groups, RBAC, Azure Policy, subscriptions, resource groups, locks, tags and cost.
- LO2: Implement and manage Azure storage — storage accounts, redundancy, access control, Blob, Azure Files and data movement.
- LO3: Deploy and manage Azure compute resources — ARM/Bicep, virtual machines, scale sets, containers and App Service.
- LO4: Implement and manage virtual networking — virtual networks, NSGs, Bastion, endpoints, DNS and load balancing.
- LO5: Monitor and maintain Azure resources — Azure Monitor, Network Watcher, Azure Backup and Azure Site Recovery.

Before You Start — Environment Setup

What you need

- An Azure subscription — an Azure free account or an instructor-provided subscription.
- Contributor access on the subscription; Global Administrator / User Administrator on the Microsoft Entra tenant for the Topic 1 identity labs.
- A modern browser for the Azure Portal (<https://portal.azure.com>) and Cloud Shell (<https://shell.azure.com>).

Launch Azure Cloud Shell

Cloud Shell is a browser terminal with the az CLI, Az PowerShell, azcopy and Bicep pre-installed — nothing to install locally. Open it from the >_ icon in the portal, accept the storage prompt on first launch, then choose Bash (for az) or PowerShell (for Az).

```
az version          # Azure CLI
```

```
az account show     # confirm your subscription
```

```
az account set --subscription "<name-or-id>" # if you have more than one
```

Conventions used in every lab

- Each lab uses its own resource group named rg-az104-labNN.
- Pick one region (e.g. eastus) and use it consistently across a lab.
- Globally-unique names (storage accounts, ACR, web apps) append a random suffix — change it if a name is taken.
- Run the Clean up step at the end of each lab: `az group delete --name rg-az104-labNN --yes --no-wait`

Topic 01 — Manage Azure Identities and Governance (20–25%)

Microsoft Entra ID · RBAC · Azure Policy · Subscriptions · Cost

Key concepts

- Microsoft Entra ID authenticates identities (users, groups, guests); Azure RBAC authorises what they may do.
- An RBAC role assignment = security principal + role definition + scope (management group / subscription / RG / resource).
- Governance is applied at a scope and inherited downward: Azure Policy, resource locks, tags, budgets.
- The resource hierarchy: management group → subscription → resource group → resource.

Lab 1 — Manage Microsoft Entra Users and Groups

Exam objective: Manage Microsoft Entra users and groups.

Goal: The learner creates cloud users, builds security and Microsoft 365 groups, manages membership and properties, and assigns licenses in Microsoft Entra ID.

What you'll build

A team of Entra users, a security group, a Microsoft 365 group, and a dynamic group (Azure services: Microsoft Entra ID, az ad, Microsoft Graph, licenses.)

Step-by-step

1. Sign in and confirm your tenant primary domain

```
DOMAIN=$(az rest --method get --url  
"https://graph.microsoft.com/v1.0/domains?$select=id,isDefault" --query  
"value[?isDefault].id" -o tsv)
```

2. Create individual cloud users

```
az ad user create --display-name "Jane Doe" --user-principal-name  
"jdoe@$DOMAIN" --password "P@ssw0rd-Lab1!" --force-change-password-next-  
sign-in true
```

3. Edit user job title, department and usage location

```
az ad user update --id "jdoe@$DOMAIN" --set jobTitle="Cloud Administrator"  
department="IT" usageLocation="SG"
```

4. Bulk create users from a list (Portal bulk create)

```
az ad user create --display-name "Amy Tan" --user-principal-name  
"atan@$DOMAIN" --password "P@ssw0rd-Lab1!" --force-change-password-next-  
sign-in true
```

5. Create a security group and a Microsoft 365 group

```
az ad group create --display-name "grp-az104-admins" --mail-nickname "grp-  
az104-admins"
```

6. Add users as members and verify membership

```
az ad group member add --group "$GROUP_ID" --member-id "$JANE_ID"
```

7. Create a dynamic-membership group driven by department

```
az rest --method post --url "https://graph.microsoft.com/v1.0/groups" --  
headers "Content-Type=application/json" --body '{"displayName":"grp-az104-  
it-dynamic","mailNickname":"grp-az104-it-  
dynamic","mailEnabled":false,"securityEnabled":true,"groupTypes":  
["DynamicMembership"],"membershipRule":"(user.department -eq  
\"IT\")","membershipRuleProcessingState":"On"}'
```

8. Inspect tenant SKUs and assign a license to a user

```
az rest --method post --url  
"https://graph.microsoft.com/v1.0/users/jdoe@$DOMAIN/assignLicense" --  
headers "Content-Type=application/json" --body '{ "addLicenses":  
[ { "skuId": "<SKU_ID>", "disabledPlans": [] } ], "removeLicenses": [] }'
```

Test it

Verify the users appear in All users, the groups list their members, the dynamic rule auto-adds IT users, and the assigned license shows on the user.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-01-*.md. Clean up when done:
az group delete --name rg-az104-lab01 --yes --no-wait

Lab 2 — Manage External (B2B Guest) Users and Configure SSPR

Exam objective: Manage external users and self-service password reset.

Goal: The learner invites a B2B guest user, reviews external collaboration and cross-tenant settings, then enables and scopes self-service password reset with authentication methods and registration.

What you'll build

A redeemed B2B guest user, a partners group, and a scoped SSPR configuration (Azure services: Microsoft Entra ID, External Identities / B2B, Password reset (SSPR), az rest.)

Step-by-step

9. Sign in and set the tenant primary-domain variable

```
DOMAIN=$(az rest --method get --url  
"https://graph.microsoft.com/v1.0/domains?$select=id,isDefault" --query  
"value[?isDefault].id" -o tsv)
```

10. Invite a B2B guest user by email

```
az rest --method post --url "https://graph.microsoft.com/v1.0/invitations"  
--headers "Content-Type=application/json" --body  
'{ "invitedUserEmailAddress": "partner.test@gmail.com",  
"invitedUserDisplayName": "Contoso Partner", "inviteRedirectUrl":  
"https://myapps.microsoft.com", "sendInvitationMessage": true }'
```

11. Verify the guest and add it to a partners group

```
az ad group member add --group "$GRP_ID" --member-id "$GUEST_ID"
```

12. Review External collaboration settings and authorization policy

```
az rest --method get --url  
"https://graph.microsoft.com/v1.0/policies/authorizationPolicy" --query  
"{allowInvitesFrom:allowInvitesFrom, guestUserRole:guestUserRoleId}"
```

13. Review Cross-tenant access (B2B) default settings

14. Enable and scope SSPR to a selected group (Portal)

15. Configure SSPR authentication methods, registration and notifications (Portal)

16. Test the reset flow; contrast SSPR with an admin reset

```
az ad user update --id "jdoe@$DOMAIN" --password "N3wP@ssw0rd-Lab2!" --  
force-change-password-next-sign-in true
```

Test it

Verify the guest shows User type = Guest with an #EXT# UPN, and a scoped test user can register methods and reset their password via <https://aka.ms/sspr>.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-02-*.md. Clean up when done:
`az group delete --name rg-az104-lab02 --yes --no-wait`

Lab 3 — Manage Access to Azure Resources with RBAC

Exam objective: Manage access to Azure resources.

Goal: The learner assigns built-in Azure roles at subscription, resource-group and resource scopes, interprets effective access with Check access, then builds and assigns a custom role.

What you'll build

Role assignments at three scopes plus a custom 'VM Restart Operator' role (Azure services: Azure RBAC (role assignments, role definitions), Resource Manager, Storage account.)

Step-by-step

17. Set variables and create a target resource group

```
az group create --name "rg-az104-lab03" --location "southeastasia"
```

18. Create a storage account to act as a scoped resource

```
az storage account create --name "$SA" --resource-group "$RG" --location "$LOCATION" --sku Standard_LRS
```

19. Explore built-in role definitions (Owner/Contributor/Reader)

```
az role definition list --name "Contributor" --query "[0].permissions[0].{actions:actions, notActions:notActions}"
```

20. Assign Reader to a group at subscription scope

```
az role assignment create --assignee-object-id "$GRP_ID" --assignee-principal-type Group --role "Reader" --scope "/subscriptions/$SUB_ID"
```

21. Assign Contributor at resource-group scope

```
az role assignment create --assignee-object-id "$JANE_ID" --assignee-principal-type User --role "Contributor" --scope "/subscriptions/$SUB_ID/resourceGroups/$RG"
```

22. Assign Storage Blob Data Reader at resource scope

```
az role assignment create --assignee-object-id "$RAJ_ID" --assignee-principal-type User --role "Storage Blob Data Reader" --scope "$SA_ID"
```

23. Interpret effective access including inherited assignments

```
az role assignment list --scope "$SA_ID" --include-inherited -o table
```

24. Create and assign a minimal custom role

```
az role definition create --role-definition /tmp/vm-restart-role.json
```

Test it

Use Check access and 'az role assignment list --include-inherited' to confirm each principal has the expected role at the right scope, and that the custom role is assignable.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-03-*.md. Clean up when done:
az group delete --name rg-az104-lab03 --yes --no-wait

Lab 4 — Implement and Manage Azure Policy

Exam objective: Implement and manage Azure Policy.

Goal: The learner assigns a built-in policy, authors a custom definition, bundles policies into an initiative, reads compliance, and remediates non-compliant resources with a managed identity.

What you'll build

An Allowed-locations assignment, a custom tag policy, a governance initiative, and a remediation task (Azure services: Azure Policy (definitions, initiatives, assignments, remediation), managed identity, Storage accounts.)

Step-by-step

25. Create a resource group as the policy scope

```
az group create --name "rg-az104-lab04" --location "southeastasia"
```

26. Assign the built-in Allowed locations policy

```
az policy assignment create --name "allowed-locations-lab04" --policy "$LOC_DEF" --scope "$SCOPE" --params '{ "listOfAllowedLocations": { "value": [ "southeastasia" ] } }'
```

27. Test the Deny effect against a disallowed region

```
az storage account create --name "stlab04deny$RANDOM" --resource-group "$RG" --location "eastus" --sku Standard_LRS
```

28. Author a custom policy definition requiring a tag

```
az policy definition create --name "require-env-tag-on-storage" --display-name "Storage accounts must have an Environment tag" --mode Indexed --rules /tmp/require-env-tag.json --subscription "$SUB_ID"
```

29. Create an initiative (policy set) and assign it

```
az policy set-definition create --name "az104-governance-baseline" --  
display-name "AZ-104 Governance Baseline" --definitions  
/tmp/initiative.json --subscription "$SUB_ID"
```

30. Trigger a scan and interpret compliance state

```
az policy state list --resource-group "$RG" --query "[].  
{resource:resourceId, state:complianceState, policy:policyDefinitionName}"  
-o table
```

31. Assign a Modify policy with managed identity and remediate

```
az policy remediation create --name "remediate-env-tag" --resource-group  
"$RG" --policy-assignment "$ASSIGN_ID"
```

32. Add an exemption to waive a specific resource

```
az policy exemption create --name "exempt-legacy-sa" --policy-assignment  
"$ASSIGN_ID" --exemption-category Waiver --scope "$SA_ID" --description  
"Legacy account exempt from Environment tag until migrated"
```

Test it

Confirm the Deny effect blocks the disallowed region, the Compliance blade shows compliant/non-compliant counts, and the remediation task tags existing resources.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-04-*.md. Clean up when done:
az group delete --name rg-az104-lab04 --yes --no-wait

Lab 5 — Manage Resource Groups, Locks and Tags

Exam objective: Manage resource groups and subscriptions.

Goal: The learner creates resource groups, deploys a resource, applies and updates tags, moves the resource between groups, and protects resources with CanNotDelete and ReadOnly locks.

What you'll build

Two resource groups, a tagged storage account moved to an archive RG, and delete/read-only locks (Azure services: Azure Resource Manager (resource groups, tags, resource locks, resource move), Storage account.)

Step-by-step

33. Create two resource groups

```
az group create --name "rg-az104-lab05" --location "southeastasia"
```

34. Deploy a storage account to tag, move and lock

```
az storage account create --name "$SA" --resource-group "$RG" --location "$LOCATION" --sku Standard_LRS
```

35. Apply tags to the resource group

```
az tag create --resource-id "/subscriptions/$(az account show --query id -o tsv)/resourceGroups/$RG" --tags CostCenter=1001 Environment=Dev Owner=angch
```

36. Merge, update and delete tags on the resource

```
az tag update --resource-id "$SA_ID" --operation Merge --tags Environment=Dev Project=AZ104
```

37. Move the resource to the archive resource group

```
az resource move --destination-group "$RG2" --ids "$SA_ID"
```

38. Apply a CanNotDelete lock on the archive RG

```
az lock create --name "lock-no-delete" --lock-type CanNotDelete --resource-group "$RG2" --notes "Protect archived resources from deletion"
```

39. Test the lock, then apply a ReadOnly lock on the resource

```
az lock create --name "lock-readonly" --lock-type ReadOnly --resource-group "$RG2" --resource-name "$SA" --resource-type "Microsoft.Storage/storageAccounts" --namespace "Microsoft.Storage"
```

40. Manage subscription-level resource providers

```
az provider register --namespace "Microsoft.Storage"
```

Test it

Verify the tags read back correctly, the resource now lives in the archive RG, and a delete attempt fails with a ScopeLocked error while the locks are in place.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-05-*.md. Clean up when done:
`az group delete --name rg-az104-lab05 --yes --no-wait`

Lab 6 — Manage Costs and Configure Management Groups

Exam objective: Manage subscriptions and governance.

Goal: The learner analyzes costs, creates a budget with alert thresholds, reviews Azure Advisor cost recommendations, and builds a management-group hierarchy to apply governance at scale.

What you'll build

A monthly budget with alerts, an action group, and an mg-az104 management-group hierarchy (Azure services: Microsoft Cost Management (Cost analysis, budgets, alerts), Azure Advisor, Management groups.)

Step-by-step

41. Set subscription and tenant variables

```
SUB_ID=$(az account show --query id -o tsv)
```

42. Analyze month-to-date cost by service

```
az costmanagement query --type ActualCost --scope "/subscriptions/$SUB_ID"
--timeframe MonthToDate --dataset-aggregation '{ "totalCost": { "name":
"Cost", "function": "Sum" } }' --dataset-grouping name="ServiceName"
type="Dimension" -o table
```

43. Create an action group for budget alert emails

```
az monitor action-group create --name "ag-budget-alerts" --resource-group
"$RG_MON" --short-name "budgetAG" --action email admin
angch@tertiaryinfotech.com
```

44. Create a monthly budget with alert thresholds

```
az consumption budget create --budget-name "budget-az104-monthly" --amount
100 --category Cost --time-grain Monthly --start-date "2026-07-01" --end-
date "2027-06-30" --notifications '{ "warning80": { "enabled": true,
"operator": "GreaterThanOrEqualTo", "threshold": 80, "contactEmails":
["angch@tertiaryinfotech.com"] } }'
```

45. Review fired cost / anomaly alerts

```
az rest --method get --url
"https://management.azure.com/subscriptions/$SUB_ID/providers/Microsoft.Cos
tManagement/alerts?api-version=2023-08-01" --query "value[].{name:name,
type:properties.definition.type, status:properties.status}" -o table
```

46. Review Azure Advisor cost recommendations

```
az advisor recommendation list --query "[?category=='Cost'].{impact:impact,
problem:shortDescription.problem, resource:resourceMetadata.resourceId}" -o
table
```

47. Create a management-group hierarchy

```
az account management-group create --name "mg-az104-prod" --display-name
"AZ104 Prod" --parent "mg-az104-root"
```

48. Move the subscription and assign inherited governance

```
az account management-group subscription add --name "mg-az104-nonprod" --
subscription "$SUB_ID"
```

Test it

Confirm the budget and action group appear with the configured thresholds, Advisor lists cost recommendations, and the subscription sits under the management-group hierarchy with the role inherited downward.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-06-*.md. Clean up when done: `az group delete --name rg-az104-lab06 --yes --no-wait`

Topic 02 — Implement and Manage Storage (15–20%)

Storage accounts · Redundancy · Access · Blob · Files · AzCopy

Key concepts

- A storage account is a globally-unique namespace holding Blobs, Files, Queues and Tables.
- Redundancy (LRS / ZRS / GRS / RA-GRS) trades cost against durability and regional failover.
- Access is controlled by keys, SAS tokens, Entra RBAC and storage firewalls / VNet rules.
- Blob access tiers (hot / cool / cold / archive), versioning, soft delete and lifecycle rules manage cost.

Lab 7 — Create and Configure Storage Accounts and Redundancy

Exam objective: Configure access to storage.

Goal: The learner creates general-purpose v2 storage accounts, switches redundancy tiers, configures encryption, and sets up object replication across accounts.

What you'll build

A source and destination storage account with an object replication policy. (Azure services: Azure Storage, Blob storage, Azure Key Vault, object replication, az storage account.)

Step-by-step

49. Create a resource group

```
az group create --name rg-az104-lab07 --location eastus
```

50. Create an LRS storage account

```
az storage account create --name $SRC --resource-group rg-az104-lab07 --location eastus --sku Standard_LRS --kind StorageV2 --access-tier Hot
```

51. Change redundancy to RA-GRS

```
az storage account update --name $SRC --resource-group rg-az104-lab07 --sku Standard_RAGRS
```

52. Inspect and set customer-managed key encryption

```
az storage account update --name $SRC --resource-group rg-az104-lab07 --encryption-key-source Microsoft.Keyvault --encryption-key-vault "https://$KV.vault.azure.net/" --encryption-key-name storagecmk
```

53. Create a destination account for replication

```
az storage account create --name $DST --resource-group rg-az104-lab07 --location westus --sku Standard_LRS --kind StorageV2
```

54. Enable versioning and change feed on both

```
az storage account blob-service-properties update --account-name $ACC --resource-group rg-az104-lab07 --enable-versioning true --enable-change-feed true
```

55. Create containers and upload a blob

```
az storage blob upload --account-name $SRC --container-name orders --name sample.txt --file sample.txt --auth-mode login
```

56. Configure an object replication policy

```
az storage account or-policy create --account-name $DST --resource-group rg-az104-lab07 --source-account $SRC --destination-account $DST --source-container orders --destination-container orders --min-creation-time '2024-01-01T00:00:00Z'
```

Test it

List blobs on the destination account and confirm the source blob has replicated asynchronously.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-07-*.md. Clean up when done: `az group delete --name rg-az104-lab07 --yes --no-wait`

Lab 8 — Configure Access to Storage: Firewalls, SAS, Access Keys

Exam objective: Configure access to storage.

Goal: The learner rotates access keys, generates account and service SAS tokens, binds a SAS to a stored access policy, locks the account behind a firewall, and grants RBAC data access.

What you'll build

A firewalled storage account secured with SAS tokens, stored access policies, and RBAC roles. (Azure services: Azure Storage, storage firewall + virtual networks, SAS, stored access policies, access keys, Azure RBAC.)

Step-by-step

57. Create the resource group and storage account

```
az storage account create --name $SA --resource-group rg-az104-lab08 --location eastus --sku Standard_LRS --kind StorageV2
```

58. Create a container and upload a blob

```
az storage blob upload --account-name $SA --container-name docs --name report.txt --file report.txt --auth-mode login
```

59. List and rotate access keys

```
az storage account keys renew --account-name $SA --resource-group rg-az104-lab08 --key primary
```

60. Generate an account SAS

```
az storage account generate-sas --account-name $SA --account-key $KEY --services b --resource-types sco --permissions rl --expiry $EXPIRY --https-only -o tsv
```

61. Generate a service SAS for one blob

```
az storage blob generate-sas --account-name $SA --account-key $KEY --container-name docs --name report.txt --permissions r --expiry $EXPIRY --https-only --full-uri -o tsv
```

62. Create a stored access policy and bind a SAS

```
az storage container policy create --account-name $SA --account-key $KEY --container-name docs --name readonly --permissions rl --expiry $POLICYEXP
```

63. Configure the storage firewall and VNet rules

```
az storage account update --name $SA --resource-group rg-az104-lab08 --default-action Deny --bypass AzureServices
```

64. Grant identity-based RBAC access

```
az role assignment create --assignee $USER --role "Storage Blob Data Contributor" --scope $SCOPE
```

Test it

List blobs with `--auth-mode login` using only your RBAC identity, with no key or SAS supplied.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-08-*.md. Clean up when done:
az group delete --name rg-az104-lab08 --yes --no-wait

Lab 9 — Configure Azure Blob Storage: Tiers, Lifecycle, Versioning

Exam objective: Configure Azure Blob Storage.

Goal: The learner creates a blob container, moves blobs between hot/cool/cold/archive tiers, enables versioning and soft delete, and authors a JSON lifecycle management policy.

What you'll build

A blob container with tiered blobs, soft delete, versioning, and an automated lifecycle policy. (Azure services: Azure Blob Storage, access tiers, blob/container soft delete, blob versioning, lifecycle management.)

Step-by-step

65. Create the resource group and storage account

```
az storage account create --name $SA --resource-group rg-az104-lab09 --  
location eastus --sku Standard_LRS --kind StorageV2 --access-tier Hot
```

66. Create a blob container

```
az storage container create --account-name $SA --name media --auth-mode  
login
```

67. Upload blobs into different tiers

```
az storage blob upload --account-name $SA --container-name media --name  
hot.txt --file hot.txt --tier Hot --auth-mode login
```

68. Change the access tier of a blob

```
az storage blob set-tier --account-name $SA --container-name media --name  
hot.txt --tier Archive --auth-mode login
```

69. Enable blob versioning

```
az storage account blob-service-properties update --account-name $SA --  
resource-group rg-az104-lab09 --enable-versioning true
```

70. Enable soft delete for blobs and containers

```
az storage account blob-service-properties update --account-name $SA --  
resource-group rg-az104-lab09 --enable-delete-retention true --delete-  
retention-days 7
```

71. Author a JSON lifecycle management policy

```
az storage account management-policy create --account-name $SA --resource-group rg-az104-lab09 --policy @policy.json
```

72. Verify the lifecycle policy

```
az storage account management-policy show --account-name $SA --resource-group rg-az104-lab09 -o json
```

Test it

Show the management policy and confirm the tier-down and expire rules match the JSON you applied.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-09-*.md. Clean up when done:
az group delete --name rg-az104-lab09 --yes --no-wait

Lab 10 — Configure Azure Files and Identity-Based Access

Exam objective: Configure Azure Files.

Goal: The learner creates an SMB file share, uploads files, takes and restores snapshots, enables share soft delete, and configures identity-based authentication with share-level RBAC.

What you'll build

An SMB file share with snapshots, soft delete, and identity-based access. (Azure services: Azure Files, share snapshots, file-share soft delete, identity-based authentication, Azure RBAC share roles.)

Step-by-step

73. Create the resource group and storage account

```
az storage account create --name $SA --resource-group rg-az104-lab10 --location eastus --sku Standard_LRS --kind StorageV2
```

74. Create a file share with a quota

```
az storage share-rm create --resource-group rg-az104-lab10 --storage-account $SA --name projects --quota 100 --access-tier TransactionOptimized
```

75. Upload a file and list the share

```
az storage file upload --account-name $SA --account-key $KEY --share-name projects --source plan.txt
```

76. Enable share soft delete

```
az storage account file-service-properties update --account-name $SA --resource-group rg-az104-lab10 --enable-delete-retention true --delete-retention-days 7
```

77. Take a share snapshot

```
az storage share snapshot --account-name $SA --account-key $KEY --name projects --query snapshot -o tsv
```

78. Restore a file from a snapshot

```
az storage file copy start --account-name $SA --account-key $KEY --source-share projects --source-path plan.txt --source-snapshot "$SNAP" --destination-share projects --destination-path plan-restored.txt
```

79. Configure identity-based authentication

```
az storage account update --name $SA --resource-group rg-az104-lab10 --enable-files-aadds true
```

80. Assign a share-level RBAC role

```
az role assignment create --assignee $USER --role "Storage File Data SMB Share Contributor" --scope "$SHARESCOPE"
```

Test it

List files inside the snapshot and confirm the restored file appears back in the live share.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-10-*.md. Clean up when done:
`az group delete --name rg-az104-lab10 --yes --no-wait`

Lab 11 — Manage Data with Storage Explorer and AzCopy

Exam objective: Configure Azure Blob Storage / Configure access to storage.

Goal: The learner provisions an account with two containers, then moves data with Azure Storage Explorer, AzCopy via SAS, and AzCopy via Entra login including a server-side copy.

What you'll build

A storage account with data transferred via Storage Explorer and AzCopy. (Azure services: Azure Storage (Blob), Azure Storage Explorer, AzCopy, SAS, Azure RBAC.)

Step-by-step

81. Create the account and two containers

```
az storage container create --account-name $SA --name source --auth-mode login
```

82. Grant yourself data-plane RBAC

```
az role assignment create --assignee $USER --role "Storage Blob Data Contributor" --scope $SCOPE
```

83. Install and connect Azure Storage Explorer

84. Upload and download with the GUI

85. Create test data and a container SAS

```
az storage container generate-sas --account-name $SA --account-key $KEY --name source --permissions racwdl --expiry $EXPIRY --https-only -o tsv
```

86. Upload a folder with AzCopy using a SAS

```
azcopy copy "$HOME/azcopydemo/*" "https://$SA.blob.core.windows.net/source?$SAS" --recursive=true
```

87. Copy account-to-account server-side

```
azcopy copy "https://$SA.blob.core.windows.net/source?$SAS" "https://$SA.blob.core.windows.net/backup?$BACKUPSAS" --recursive=true
```

88. Use AzCopy with Entra login and sync

```
azcopy sync "$HOME/azcopydemo" "https://$SA.blob.core.windows.net/source" --recursive=true
```

Test it

Run `azcopy list` on the source container URL and confirm the uploaded and synced files are present.

Note: Full commands (Portal + CLI + PowerShell) are in `labs/lab-11-*.md`. Clean up when done: `az group delete --name rg-az104-lab11 --yes --no-wait`

Topic 03 — Deploy and Manage Azure Compute Resources (20–25%)

ARM & Bicep · VMs · Scale Sets · Containers · App Service

Key concepts

- Infrastructure as code with ARM templates and Bicep makes deployments repeatable and reviewable.
- Choose the least-effort compute: VM → VM Scale Set → containers (ACI / Container Apps) → App Service.

- Availability zones and sets, managed disks and VM sizes determine resilience and performance.
- App Service and Container Apps are fully-managed PaaS for web apps, APIs and microservices.

Lab 12 — Automate Deployment with ARM Templates and Bicep

Exam objective: Automate deployment of resources by using ARM templates or Bicep files.

Goal: Read, edit, and deploy resources with ARM templates and Bicep, then export and decompile between the two formats.

What you'll build

A storage account deployed from both an ARM template and a Bicep file. (Azure services: ARM, Bicep, az deployment, Azure Storage, Cloud Shell.)

Step-by-step

89. Create the resource group

```
az group create --name rg-az104-lab12 --location eastus
```

90. Interpret an ARM template's parameters/resources/outputs

91. Modify the ARM template to add SKU and harden TLS

92. Deploy the ARM template to the resource group

```
az deployment group create --resource-group rg-az104-lab12 --template-file storage.json --parameters storageAccountName=$SANAME skuName=Standard_LRS
```

93. Author and edit a Bicep file

94. Compile the Bicep file to ARM JSON

```
az bicep build --file storage.bicep
```

95. Deploy the Bicep file

```
az deployment group create --resource-group rg-az104-lab12 --template-file storage.bicep --parameters storageAccountName=$SANAME2 skuName=Standard_ZRS
```

96. Export the deployment as an ARM template

```
az group export --name rg-az104-lab12 > exported-rg.json
```

97. Decompile the ARM template to Bicep

```
az bicep decompile --file exported-rg.json
```

Test it

Confirm both storage accounts exist and the exported template decompiles to Bicep.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-12-*.md. Clean up when done:
`az group delete --name rg-az104-lab12 --yes --no-wait`

Lab 13 — Create a Virtual Machine, Encryption at Host, and Moving VMs

Exam objective: Create and configure virtual machines.

Goal: Deploy a Linux VM with encryption at host enabled, then move it across resource groups, subscriptions, and regions.

What you'll build

A Linux VM with host-based encryption, moved between resource groups and regions. (Azure services: Azure Virtual Machines, Managed Disks, Azure Resource Mover, ARM.)

Step-by-step

98. Create source and destination resource groups

```
az group create --name rg-az104-lab13 --location eastus
```

99. Register the encryption-at-host feature

```
az feature register --namespace Microsoft.Compute --name EncryptionAtHost
```

100. Create a VM with encryption at host

```
az vm create --resource-group rg-az104-lab13 --name vm-lab13 --image Ubuntu2204 --size Standard_B1s --admin-username azureuser --generate-ssh-keys --encryption-at-host --public-ip-sku Standard
```

101. Verify encryption at host on the VM

```
az vm show -g rg-az104-lab13 -n vm-lab13 --query securityProfile.encryptionAtHost
```

102. Move the VM to another resource group

```
az resource move --destination-group rg-az104-lab13-dest --ids $VMID
```

103. Move the VM to another subscription

```
az resource move --destination-group rg-az104-lab13-dest --destination-subscription-id $TARGET_SUB --ids $VMID
```

104. Move the VM to another region with Resource Mover

```
az resource-mover move-collection create --name mc-lab13 --resource-group rg-az104-lab13 --source-region eastus --target-region westus2 --location eastus2euap --identity type=SystemAssigned
```

Test it

Confirm encryptionAtHost is true and the VM appears in the destination resource group.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-13-*.md. Clean up when done:
`az group delete --name rg-az104-lab13 --yes --no-wait`

Lab 14 — VM Sizes, Disks, and Availability (Zones & Sets)

Exam objective: Manage virtual machines.

Goal: Resize a VM, attach and expand a managed data disk, and deploy VMs into an availability zone and an availability set.

What you'll build

A resized VM with an expanded data disk plus zonal and availability-set VMs. (Azure services: Azure Virtual Machines, Managed Disks, Availability Zones, Availability Sets.)

Step-by-step

105. Create the resource group and a base VM

```
az vm create --resource-group rg-az104-lab14 --name vm-lab14 --image Ubuntu2204 --size Standard_B1s --admin-username azureuser --generate-ssh-keys --public-ip-sku Standard
```

106. List available and resizable VM sizes

```
az vm list-vm-resize-options --resource-group rg-az104-lab14 --name vm-lab14 --output table
```

107. Resize the VM

```
az vm resize --resource-group rg-az104-lab14 --name vm-lab14 --size Standard_B2s
```

108. Create and attach a managed data disk

```
az vm disk attach --resource-group rg-az104-lab14 --vm-name vm-lab14 --name datadisk-lab14
```

109. Expand the data disk

```
az disk update --resource-group rg-az104-lab14 --name datadisk-lab14 --size-gb 64
```

110. Deploy a VM into an availability zone

```
az vm create --resource-group rg-az104-lab14 --name vm-zone1 --image Ubuntu2204 --size Standard_B1s --zone 1 --admin-username azureuser --generate-ssh-keys --public-ip-sku Standard
```

111. Create an availability set

```
az vm availability-set create --resource-group rg-az104-lab14 --name avset-lab14 --platform-fault-domain-count 2 --platform-update-domain-count 5
```

112. Deploy VMs into the availability set

```
az vm create -g rg-az104-lab14 -n vm-set-1 --availability-set avset-lab14 --image Ubuntu2204 --size Standard_B1s --admin-username azureuser --generate-ssh-keys
```

Test it

Confirm the resized VM, the 64 GiB disk, the zonal VM, and both availability-set VMs exist.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-14-*.md. Clean up when done: az group delete --name rg-az104-lab14 --yes --no-wait

Lab 15 — Deploy and Configure a Virtual Machine Scale Set (VMSS) with Autoscale

Exam objective: Deploy and configure a Virtual Machine Scale Set.

Goal: Create a VMSS behind a load balancer and attach autoscale rules that add and remove instances based on CPU usage.

What you'll build

A Flexible VMSS with CPU-based scale-out and scale-in autoscale rules. (Azure services: Azure Virtual Machine Scale Sets, Azure Load Balancer, Azure Monitor Autoscale.)

Step-by-step

113. Create the resource group

```
az group create --name rg-az104-lab15 --location eastus
```

114. Create the scale set behind a load balancer

```
az vmss create --resource-group rg-az104-lab15 --name vmss-lab15 --orchestration-mode Flexible --image Ubuntu2204 --vm-sku Standard_B1s --instance-count 2 --admin-username azureuser --generate-ssh-keys --load-balancer vmss-lb --upgrade-policy-mode automatic
```


115. View and manually scale instances

```
az vmss scale --resource-group rg-az104-lab15 --name vmss-lab15 --new-capacity 3
```

116. Create an autoscale profile with min/max/default

```
az monitor autoscale create --resource-group rg-az104-lab15 --resource vmss-lab15 --resource-type Microsoft.Compute/virtualMachineScaleSets --name autoscale-lab15 --min-count 2 --max-count 10 --count 2
```

117. Add a scale-out rule when CPU is high

```
az monitor autoscale rule create --resource-group rg-az104-lab15 --autoscale-name autoscale-lab15 --condition "Percentage CPU > 70 avg 5m" --scale out 1
```

118. Add a scale-in rule when CPU is low

```
az monitor autoscale rule create --resource-group rg-az104-lab15 --autoscale-name autoscale-lab15 --condition "Percentage CPU < 30 avg 5m" --scale in 1
```

119. Review the autoscale configuration

```
az monitor autoscale rule list --resource-group rg-az104-lab15 --autoscale-name autoscale-lab15 --output table
```

120. Update the instance SKU and roll to instances

```
az vmss update --resource-group rg-az104-lab15 --name vmss-lab15 --set sku.name=Standard_B2s
```

Test it

Confirm both scale rules and the min/max bounds appear in the autoscale settings.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-15-*.md. Clean up when done:
`az group delete --name rg-az104-lab15 --yes --no-wait`

Lab 16 — Containers: ACR, Container Instances, and Container Apps

Exam objective: Create and configure containers.

Goal: Build and push an image to Azure Container Registry, run it in Container Instances, then deploy and scale it in Container Apps.

What you'll build

An image in ACR running in both Container Instances and a scalable Container App.
(Azure services: Azure Container Registry, Azure Container Instances, Azure Container Apps, Log Analytics.)

Step-by-step

121. Create the resource group

```
az group create --name rg-az104-lab16 --location eastus
```

122. Create an Azure Container Registry

```
az acr create --resource-group rg-az104-lab16 --name $ACR --sku Basic --  
admin-enabled true
```

123. Build and push an image with ACR Tasks

```
az acr build --registry $ACR --image webapp:v1 https://github.com/Azure-  
Samples/acr-build-helloworld-node.git
```

124. Provision a container with Container Instances

```
az container create --resource-group rg-az104-lab16 --name aci-lab16 --  
image $ACR_SERVER/webapp:v1 --registry-login-server $ACR_SERVER --registry-  
username $ACR_USER --registry-password $ACR_PASS --dns-name-label  
acilab16$RANDOM --ports 80 --cpu 1 --memory 1.5
```

125. Create a Container Apps environment

```
az containerapp env create --name aca-env-lab16 --resource-group rg-az104-  
lab16 --location eastus
```

126. Provision a container with Container Apps

```
az containerapp create --name aca-lab16 --resource-group rg-az104-lab16 --  
environment aca-env-lab16 --image $ACR_SERVER/webapp:v1 --registry-server  
$ACR_SERVER --registry-username $ACR_USER --registry-password $ACR_PASS --  
target-port 80 --ingress external --cpu 0.5 --memory 1.0Gi
```

127. Manage Container App sizing and scaling

```
az containerapp update --name aca-lab16 --resource-group rg-az104-lab16 --  
min-replicas 0 --max-replicas 5 --scale-rule-name http-rule --scale-rule-  
type http --scale-rule-http-concurrency 50
```

128. Manage registry images and usage

```
az acr repository delete --name $ACR --image webapp:v1 --yes
```

Test it

Browse the ACI and Container App FQDNs and confirm the web app responds.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-16-*.md. Clean up when done:
`az group delete --name rg-az104-lab16 --yes --no-wait`

Lab 17 — Azure App Service: Plans, Scaling, TLS, Custom Domains, Backup, Networking, Slots

Exam objective: Create and configure Azure App Service.

Goal: Create an App Service plan and web app, then configure scaling, TLS, custom domains, backup, networking, and deployment slots.

What you'll build

A Standard-tier web app with autoscale, TLS, backup, VNet integration, and a staging slot. (Azure services: Azure App Service, App Service plans, Azure Monitor Autoscale, Managed Certificates, VNet integration.)

Step-by-step

129. Create the resource group

```
az group create --name rg-az104-lab17 --location eastus
```

130. Provision a Standard App Service plan

```
az appservice plan create --resource-group rg-az104-lab17 --name plan-lab17 --sku S1 --is-linux
```

131. Create an App Service web app

```
az webapp create --resource-group rg-az104-lab17 --plan plan-lab17 --name $APP --runtime "NODE:20-lts"
```

132. Configure scale up, scale out, and autoscale

```
az monitor autoscale create -g rg-az104-lab17 --resource $PLANID --name autoscale-lab17 --min-count 1 --max-count 5 --count 1
```

133. Map a custom DNS name

```
az webapp config hostname add --resource-group rg-az104-lab17 --webapp-name $APP --hostname www.contoso.com
```

134. Create and bind a managed TLS certificate

```
az webapp config ssl bind --resource-group rg-az104-lab17 --name $APP --certificate-thumbprint $THUMB --ssl-type SNI
```

135. Configure scheduled backups to storage

```
az webapp config backup update --resource-group rg-az104-lab17 --webapp-name $APP --container-url "$CONTAINER_URL" --frequency 1d --retain-one true --retention 30
```

136. Configure VNet integration and access restrictions

```
az webapp vnet-integration add -g rg-az104-lab17 -n $APP --vnet vnet-lab17 --subnet appsvc-subnet
```

137. Create a deployment slot and swap

```
az webapp deployment slot swap --resource-group rg-az104-lab17 --name $APP --slot staging --target-slot production
```

Test it

Confirm the app serves over HTTPS with TLS 1.2 and the staging slot swaps into production.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-17-*.md. Clean up when done: `az group delete --name rg-az104-lab17 --yes --no-wait`

Topic 04 — Implement and Manage Virtual Networking (15–20%)

VNets · NSGs & ASGs · Bastion · Endpoints · DNS · Load Balancing

Key concepts

- A virtual network (VNet) is your private network in Azure, divided into subnets; peering joins VNets.
- NSGs and ASGs filter traffic; effective security rules show what actually applies to a NIC.
- Azure Bastion gives browser RDP/SSH with no public IP; service/private endpoints lock PaaS to the VNet.
- Azure DNS resolves names; load balancers distribute traffic across a backend pool with health probes.

Lab 18 — Virtual Networks, Subnets & VNet Peering

Exam objective: Configure virtual networks.

Goal: Create two virtual networks with non-overlapping address spaces and subnets, then peer them so resources communicate over the Azure backbone without a VPN or public IPs.

What you'll build

Two peered VNets (hub and spoke) in Connected state. (Azure services: Virtual Network, Subnets, VNet Peering, Network Watcher, az network vnet.)

Step-by-step

138. Create the resource group

```
az group create --name rg-az104-lab18 --location eastus
```

139. Create first VNet vnet-hub with a subnet

```
az network vnet create --resource-group rg-az104-lab18 --name vnet-hub --address-prefixes 10.10.0.0/16 --subnet-name snet-hub-app --subnet-prefixes 10.10.1.0/24 --location eastus
```

140. Create second VNet vnet-spoke with a subnet

```
az network vnet create --resource-group rg-az104-lab18 --name vnet-spoke --address-prefixes 10.20.0.0/16 --subnet-name snet-spoke-app --subnet-prefixes 10.20.1.0/24 --location eastus
```

141. Add a second subnet to the hub VNet

```
az network vnet subnet create --resource-group rg-az104-lab18 --vnet-name vnet-hub --name snet-hub-mgmt --address-prefixes 10.10.2.0/24
```

142. Create the peering from hub to spoke

```
az network vnet peering create --resource-group rg-az104-lab18 --name hub-to-spoke --vnet-name vnet-hub --remote-vnet vnet-spoke --allow-vnet-access
```

143. Create the reverse peering from spoke to hub

```
az network vnet peering create --resource-group rg-az104-lab18 --name spoke-to-hub --vnet-name vnet-spoke --remote-vnet vnet-hub --allow-vnet-access
```

144. Verify the peering state is Connected

```
az network vnet peering show --resource-group rg-az104-lab18 --vnet-name vnet-hub --name hub-to-spoke --query "{name:name, state:peeringState}" -o table
```

145. Deploy test VMs and check connectivity

```
az network watcher test-connectivity --resource-group rg-az104-lab18 --source-resource vm-hub --dest-resource vm-spoke --dest-port 22
```

Test it

Both peerings show peeringState = Connected and test-connectivity across the peering returns Reachable.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-18-*.md. Clean up when done:
az group delete --name rg-az104-lab18 --yes --no-wait

Lab 19 — Public IPs, User-Defined Routes & Connectivity Troubleshooting

Exam objective: Configure IP addressing and routing.

Goal: Allocate a static Standard public IP, build a route table with a user-defined route forcing traffic through a virtual appliance, associate it to a subnet, then troubleshoot the resulting effective routes.

What you'll build

A route table with a 0.0.0.0/0 UDR associated to a workload subnet. (Azure services: Public IP address, Route Table (UDR), Virtual Network, Network Watcher, NIC, az network route-table.)

Step-by-step

146. Create the resource group and a VNet

```
az network vnet create --resource-group rg-az104-lab19 --name vnet-lab19 --address-prefixes 10.30.0.0/16 --subnet-name snet-workload --subnet-prefixes 10.30.1.0/24
```

147. Create a static Standard SKU public IP

```
az network public-ip create --resource-group rg-az104-lab19 --name pip-web-static --sku Standard --allocation-method Static --version IPv4
```

148. Inspect the allocated address

```
az network public-ip show --resource-group rg-az104-lab19 --name pip-web-static --query "{ip:ipAddress, sku:sku.name}" -o table
```

149. Create a route table

```
az network route-table create --resource-group rg-az104-lab19 --name rt-workload --location eastus
```

150. Add a UDR to a virtual appliance

```
az network route-table route create --resource-group rg-az104-lab19 --route-table-name rt-workload --name route-to-nva --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance --next-hop-ip-address 10.30.1.4
```

151. Associate the route table to the subnet

```
az network vnet subnet update --resource-group rg-az104-lab19 --vnet-name vnet-lab19 --name snet-workload --route-table rt-workload
```

152. Deploy a test VM with the static public IP

```
az vm create --resource-group rg-az104-lab19 --name vm-web --image Ubuntu2204 --size Standard_B1s --vnet-name vnet-lab19 --subnet snet-workload --public-ip-address pip-web-static --admin-username azureuser --generate-ssh-keys
```

153. View the effective routes on the NIC

```
az network nic show-effective-route-table --resource-group rg-az104-lab19 --name "$NIC" -o table
```

154. Troubleshoot connectivity with Network Watcher

```
az network watcher test-connectivity --resource-group rg-az104-lab19 --source-resource vm-web --dest-address 13.107.42.14 --dest-port 443
```

Test it

The NIC effective routes show a User-source 0.0.0.0/0 route to VirtualAppliance, and test-connectivity returns Unreachable confirming the UDR diverts traffic.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-19-*.md. Clean up when done: `az group delete --name rg-az104-lab19 --yes --no-wait`

Lab 20 — Network Security Groups, Application Security Groups & Effective Rules

Exam objective: Secure network traffic.

Goal: Create an NSG with prioritized custom rules, group VMs by role using application security groups, reference those ASGs in NSG rules, then read the effective security rules Azure computes for a NIC.

What you'll build

An NSG with ASG-based allow/deny rules associated to a subnet. (Azure services: Network Security Group, Application Security Group, Virtual Network, NIC, Network Watcher, `az network nsg`.)

Step-by-step

155. Create the resource group, VNet and subnet

```
az network vnet create --resource-group rg-az104-lab20 --name vnet-lab20 --address-prefixes 10.40.0.0/16 --subnet-name snet-app --subnet-prefixes 10.40.1.0/24
```

156. Create application security groups

```
az network asg create --resource-group rg-az104-lab20 --name asg-web --location eastus
```

157. Create the network security group

```
az network nsg create --resource-group rg-az104-lab20 --name nsg-app --location eastus
```

158. Allow HTTPS from internet to the web ASG

```
az network nsg rule create --resource-group rg-az104-lab20 --nsg-name nsg-app --name Allow-HTTPS-Web --priority 100 --direction Inbound --access Allow --protocol Tcp --source-address-prefixes Internet --destination-asgs asg-web --destination-port-ranges 443
```

159. Allow SQL only from web ASG to db ASG

```
az network nsg rule create --resource-group rg-az104-lab20 --nsg-name nsg-app --name Allow-SQL-Web-to-DB --priority 110 --direction Inbound --access Allow --protocol Tcp --source-asgs asg-web --destination-asgs asg-db --destination-port-ranges 1433
```

160. Explicitly deny everything else inbound

```
az network nsg rule create --resource-group rg-az104-lab20 --nsg-name nsg-app --name Deny-All-Inbound --priority 4000 --direction Inbound --access Deny --protocol '*' --source-address-prefixes '*' --destination-address-prefixes '*' --destination-port-ranges '*'
```

161. Associate the NSG to the subnet

```
az network vnet subnet update --resource-group rg-az104-lab20 --vnet-name vnet-lab20 --name subnet-app --network-security-group nsg-app
```

162. Deploy VMs and place NICs in the ASGs

```
az network nic ip-config update -g rg-az104-lab20 --nic-name "$WEB_NIC" -n ipconfig1 --application-security-groups asg-web
```

163. Evaluate the effective security rules

```
az network nic list-effective-nsg --resource-group rg-az104-lab20 --name "$WEB_NIC" -o table
```

Test it

IP flow verify on vm-web returns Allow by Allow-HTTPS-Web for port 443 and Deny for port 22.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-20-*.md. Clean up when done: `az group delete --name rg-az104-lab20 --yes --no-wait`

Lab 21 — Azure Bastion, Service Endpoints & Private Endpoints

Exam objective: Provide secure access to PaaS and VMs.

Goal: Deploy Azure Bastion for browser-based RDP/SSH without VM public IPs, lock a storage account to a subnet with a service endpoint, then bring that storage into the VNet with a private endpoint and private DNS.

What you'll build

Azure Bastion plus a storage account reachable via a private endpoint. (Azure services: Azure Bastion, Virtual Network, Public IP, Service Endpoints, Private Endpoint, Private DNS Zone, Azure Storage.)

Step-by-step

164. Create the resource group and VNet

```
az network vnet create --resource-group rg-az104-lab21 --name vnet-lab21 --address-prefixes 10.50.0.0/16 --subnet-name snet-workload --subnet-prefixes 10.50.1.0/24
```

165. Add the AzureBastionSubnet (/26 or larger)

```
az network vnet subnet create --resource-group rg-az104-lab21 --vnet-name vnet-lab21 --name AzureBastionSubnet --address-prefixes 10.50.2.0/26
```

166. Create a Standard public IP for Bastion

```
az network public-ip create --resource-group rg-az104-lab21 --name pip-bastion --sku Standard --allocation-method Static
```

167. Deploy Azure Bastion

```
az network bastion create --resource-group rg-az104-lab21 --name bastion-lab21 --vnet-name vnet-lab21 --public-ip-address pip-bastion --location eastus --sku Standard
```

168. Deploy a VM with no public IP for Bastion

```
az vm create --resource-group rg-az104-lab21 --name vm-jump --image Ubuntu2204 --size Standard_B1s --vnet-name vnet-lab21 --subnet snet-workload --public-ip-address "" --admin-username azureuser --generate-ssh-keys
```

169. Create a storage account for endpoint tests

```
az storage account create --resource-group rg-az104-lab21 --name $STG --sku Standard_LRS --kind StorageV2 --location eastus
```

170. Enable a service endpoint on the subnet

```
az network vnet subnet update --resource-group rg-az104-lab21 --vnet-name vnet-lab21 --name snet-workload --service-endpoints Microsoft.Storage
```

171. Restrict the storage account to that subnet

```
az storage account network-rule add --resource-group rg-az104-lab21 --account-name $STG --vnet-name vnet-lab21 --subnet snet-workload
```

172. Create a private endpoint and integrate DNS

```
az network private-endpoint create --resource-group rg-az104-lab21 --name pe-storage-blob --vnet-name vnet-lab21 --subnet snet-workload --private-connection-resource-id "$STG_ID" --group-id blob --connection-name pe-storage-conn
```

Test it

From vm-jump via Bastion, nslookup of the storage blob endpoint resolves to a 10.50.x.x private IP instead of a public address.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-21-*.md. Clean up when done: `az group delete --name rg-az104-lab21 --yes --no-wait`

Lab 22 — Azure DNS & Load Balancing

Exam objective: Configure name resolution and load balancing.

Goal: Host a public DNS zone with A and CNAME records, then build a public Standard load balancer with a backend pool, health probe and rule across two VMs, and troubleshoot uneven traffic distribution.

What you'll build

A public DNS zone plus a Standard load balancer distributing across two VMs. (Azure services: Azure DNS, Azure Load Balancer (Standard), Public IP, Virtual Network, Backend Pool, Health Probe, Network Watcher.)

Step-by-step

173. Create the resource group and VNet

```
az network vnet create --resource-group rg-az104-lab22 --name vnet-lab22 --address-prefixes 10.60.0.0/16 --subnet-name snet-backend --subnet-prefixes 10.60.1.0/24
```

174. Create a public Azure DNS zone

```
az network dns zone create --resource-group rg-az104-lab22 --name contoso-az104-demo.com
```

175. Add A and CNAME records to the zone

```
az network dns record-set a add-record --resource-group rg-az104-lab22 --  
zone-name contoso-az104-demo.com --record-set-name www --ipv4-address  
20.1.2.3
```

176. Create a Standard public IP for the LB

```
az network public-ip create --resource-group rg-az104-lab22 --name pip-lb  
--sku Standard --allocation-method Static
```

177. Create the public load balancer with frontend

```
az network lb create --resource-group rg-az104-lab22 --name lb-web --sku  
Standard --public-ip-address pip-lb --frontend-ip-name fe-web --backend-  
pool-name pool-web
```

178. Add a health probe

```
az network lb probe create --resource-group rg-az104-lab22 --lb-name lb-web  
--name probe-http --protocol Tcp --port 80 --interval 5
```

179. Add a load-balancing rule

```
az network lb rule create --resource-group rg-az104-lab22 --lb-name lb-web  
--name rule-http --protocol Tcp --frontend-port 80 --backend-port 80 --  
frontend-ip-name fe-web --backend-pool-name pool-web --probe-name probe-  
http
```

180. Deploy two backend VMs into the pool

```
az network nic ip-config address-pool add --resource-group rg-az104-lab22  
--nic-name "$NIC" --ip-config-name ipconfig1 --lb-name lb-web --address-  
pool pool-web
```

181. Test and troubleshoot load balancing

```
az monitor metrics list --resource $(az network lb show -g rg-az104-lab22  
-n lb-web --query id -o tsv) --metric DipAvailability VipAvailability -o  
table
```

Test it

Browsing the LB frontend IP repeatedly alternates responses between vm-web1 and vm-web2, confirming traffic distribution.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-22-*.md. Clean up when done:
az group delete --name rg-az104-lab22 --yes --no-wait

Topic 05 — Monitor and Maintain Azure Resources (10–15%)

Azure Monitor · Network Watcher · Backup · Site Recovery

Key concepts

- Azure Monitor collects metrics and logs (Log Analytics + KQL) and raises alerts via action groups.
- Network Watcher diagnoses connectivity (IP flow verify, next hop, connection monitor).
- Azure Backup protects data with point-in-time restore from a Recovery Services vault.
- Azure Site Recovery replicates workloads to a second region for disaster-recovery failover.

Lab 23 — Azure Monitor: Metrics, Logs, KQL, and Alerts

Exam objective: Monitor resources with Azure Monitor.

Goal: Build the full Azure Monitor loop: a Log Analytics workspace, diagnostic settings, metrics, KQL queries, and alerts.

What you'll build

A Log Analytics workspace with diagnostic data, KQL queries, and metric/activity/processing alert rules. (Azure services: Azure Monitor, Log Analytics workspace, Metrics, Alerts, Action Groups, Alert Processing Rules, VM Insights.)

Step-by-step

182. Create resource group and Log Analytics workspace

```
az monitor log-analytics workspace create --resource-group "$RG" --  
workspace-name "$WS" --location "$LOCATION" --retention-time 30
```

183. Deploy a monitored VM and storage account

```
az vm create --resource-group "$RG" --name "vm-az104-lab23" --image  
Ubuntu2204 --size Standard_B1s --admin-username azureuser --generate-ssh-  
keys --public-ip-sku Standard
```

184. Read platform metrics such as Percentage CPU

```
az monitor metrics list --resource "$VM_ID" --metric "Percentage CPU" --  
aggregation Average --interval PT5M
```

185. Route logs via diagnostic settings to the workspace

```
az monitor diagnostic-settings create --name "diag-to-law" --resource  
"$STORAGE_ID/blobServices/default" --workspace "$WS_ID" --logs
```

```
'[{"category":"StorageRead","enabled":true}]' --metrics  
'[{"category":"Transaction","enabled":true}]'
```

186. Enable VM Insights (Azure Monitor Agent + DCR)

```
az vm extension set --resource-group "$RG" --vm-name "vm-az104-lab23" --  
name AzureMonitorLinuxAgent --publisher Microsoft.Azure.Monitor --enable-  
auto-upgrade true
```

187. Query logs with KQL against Heartbeat/Perf/AzureActivity

```
az monitor log-analytics query --workspace "$WS_GUID" --analytics-query  
"Heartbeat | summarize LastHeartbeat = max(TimeGenerated) by Computer" --  
output table
```

188. Create an action group for notifications

```
az monitor action-group create --resource-group "$RG" --name "ag-az104-  
lab23" --short-name "az104ag" --action email admin  
angch@tertiaryinfotech.com
```

189. Create a metric alert on high CPU

```
az monitor metrics alert create --name "alert-vm-highcpu" --resource-group  
"$RG" --scopes "$VM_ID" --condition "avg Percentage CPU > 80" --window-size  
5m --evaluation-frequency 1m --severity 3 --action "$AG_ID"
```

190. Create an alert processing rule across the RG

```
az monitor alert-processing-rule create --name "apr-add-ag" --resource-  
group "$RG" --scopes "/subscriptions/$SUB_ID/resourceGroups/$RG" --rule-  
type AddActionGroups --action-groups "$AG_ID"
```

Test it

In Monitor → Logs the KQL queries return data and the metric/activity/processing alert rules appear under Monitor → Alerts.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-23-*.md. Clean up when done:
az group delete --name rg-az104-lab23 --yes --no-wait

Lab 24 — Azure Network Watcher and Connection Monitor

Exam objective: Monitor networks with Azure Network Watcher.

Goal: Use Network Watcher's diagnostic tools to verify traffic rules, trace routing, capture packets, and continuously monitor reachability between VMs.

What you'll build

Two VMs diagnosed with IP flow verify, next hop, NSG flow logs, packet capture, and a Connection Monitor. (Azure services: Azure Network Watcher, IP Flow Verify, Next Hop, NSG Diagnostics, Packet Capture, Connection Monitor, VNet, NSG, VMs.)

Step-by-step

191. Create resource group and virtual network

```
az network vnet create --resource-group "$RG" --name "vnet-az104-lab24" --  
address-prefix 10.10.0.0/16 --subnet-name "subnet-app" --subnet-prefix  
10.10.1.0/24
```

192. Deploy two VMs with an NSG allowing SSH/HTTP

```
az vm create -g "$RG" -n "vm-web" --image Ubuntu2204 --size Standard_B1s --  
vnet-name "vnet-az104-lab24" --subnet "subnet-app" --admin-username  
azureuser --generate-ssh-keys --public-ip-sku Standard --nsg "nsg-web"
```

193. Run IP flow verify to check a 5-tuple

```
az network watcher test-ip-flow --vm "$VMWEB_ID" --direction Inbound --  
protocol TCP --local "10.10.1.4:80" --remote "13.68.0.1:50000"
```

194. Trace routing with next hop

```
az network watcher show-next-hop --resource-group "$RG" --vm "vm-web" --  
source-ip 10.10.1.4 --dest-ip 8.8.8.8
```

195. View effective NSG rules on the NIC

```
az network nic list-effective-nsg --ids "$NIC_ID" -o jsonc
```

196. Enable NSG flow logs to a storage account

```
az network watcher flow-log create --location "$LOCATION" --name "flowlog-  
nsg-web" --nsg "$NSG_ID" --storage-account "$STORAGE_ID" --retention 7 --  
enabled true
```

197. Capture packets on the VM for offline analysis

```
az network watcher packet-capture create --resource-group "$RG" --vm "vm-  
web" --name "pcap-vm-web" --storage-account "$STORAGE" --time-limit 60
```

198. Run a point-in-time connection troubleshoot

```
az network watcher test-connectivity --resource-group "$RG" --source-  
resource "vm-web" --dest-address "$VMAPP_IP" --dest-port 22
```

199. Configure a continuous Connection Monitor

```
az network watcher connection-monitor create --name "cm-web-to-app" --  
location "$LOCATION" --endpoint-source-name "src-vm-web" --endpoint-source-  
resource-id "$VMWEB_ID" --endpoint-dest-name "dst-vm-app" --endpoint-dest-
```

```
address "$VMAPP_IP" --test-config-name "tcp-22" --protocol Tcp --tcp-port 22 --frequency 30
```

Test it

IP flow verify returns Allow/Deny with the deciding rule, and Connection Monitor shows ongoing reachability results between vm-web and vm-app.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-24-*.md. Clean up when done: `az group delete --name rg-az104-lab24 --yes --no-wait`

Lab 25 — Azure Backup: Recovery Services Vault, Policies, Backup and Restore

Exam objective: Implement backup and recovery.

Goal: Create a Recovery Services vault, author a custom backup policy, protect a VM, run backups, and perform file-level and full VM restores.

What you'll build

A Recovery Services vault protecting a VM with a custom policy, recovery points, and configured backup reports/alerts. (Azure services: Recovery Services vault, Azure Backup, Backup policy, Virtual Machine, Backup center / Backup reports, Azure Monitor alerts.)

Step-by-step

200. Create resource group and Recovery Services vault

```
az backup vault create --resource-group "$RG" --name "$VAULT" --location "$LOCATION"
```

201. Set backup storage redundancy to GRS

```
az backup vault backup-properties set --resource-group "$RG" --name "$VAULT" --backup-storage-redundancy GeoRedundant
```

202. Deploy a VM to protect

```
az vm create --resource-group "$RG" --name "vm-az104-lab25" --image Ubuntu2204 --size Standard_B1s --admin-username azureuser --generate-ssh-keys --public-ip-sku Standard
```

203. Create a custom daily backup policy

```
az backup policy create --resource-group "$RG" --vault-name "$VAULT" --name "pol-daily-30d" --backup-management-type AzureIaaSVM --policy @policy.json
```

204. Enable backup protection on the VM

```
az backup protection enable-for-vm --resource-group "$RG" --vault-name "$VAULT" --vm "vm-az104-lab25" --policy-name "pol-daily-30d"
```

205. Run an on-demand backup

```
az backup protection backup-now --resource-group "$RG" --vault-name "$VAULT" --container-name "vm-az104-lab25" --item-name "vm-az104-lab25" --backup-management-type AzureIaaSVM --retain-until 31-07-2026
```

206. Restore individual files via ILR mount

```
az backup restore files mount-rp --resource-group "$RG" --vault-name "$VAULT" --container-name "vm-az104-lab25" --item-name "vm-az104-lab25" --rp-name "$RP"
```

207. Restore the full VM disks to staging

```
az backup restore restore-disks --resource-group "$RG" --vault-name "$VAULT" --container-name "vm-az104-lab25" --item-name "vm-az104-lab25" --rp-name "$RP" --storage-account "$STORAGE" --target-resource-group "$RG"
```

208. Configure vault diagnostics for backup reports/alerts

```
az monitor diagnostic-settings create --name "vault-diag" --resource "$VAULT_ID" --workspace "$WS_ID" --logs ' [{"category": "CoreAzureBackup", "enabled": true}, {"category": "AddonAzureBackupJobs", "enabled": true} ] '
```

Test it

az backup item list shows the VM protected with recovery points, and the restore jobs complete successfully in Backup Jobs.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-25-*.md. Clean up when done: az group delete --name rg-az104-lab25 --yes --no-wait

Lab 26 — Azure Site Recovery: Replicate, Fail Over, and Interpret Health

Exam objective: Implement disaster recovery.

Goal: Enable Azure-to-Azure replication for a VM, run test and real failovers with re-protect and failback, and interpret replication health.

What you'll build

A VM replicated to a secondary region via ASR with test/real failover, failback, and a Recovery Plan. (Azure services: Recovery Services vault (Site Recovery), Azure Site Recovery (Azure-to-Azure), source VM, source/target VNets, cache storage account.)

Step-by-step

209. Create resource group, source VM, and vault

```
az backup vault create --resource-group "$RG" --name "$VAULT" --location "$PRIMARY"
```

210. Pre-create DR resource group, VNet, and cache storage

```
az network vnet create --resource-group "rg-az104-lab26-dr" --name "vnet-dr" --location "$SECONDARY" --address-prefix 10.20.0.0/16 --subnet-name "default" --subnet-prefix 10.20.1.0/24
```

211. Enable Site Recovery replication for the VM

212. Adjust the replication policy (snapshot frequency/retention)

213. Interpret replication health, status, and RPO

```
az resource list --resource-group "$RG" --resource-type "Microsoft.RecoveryServices/vaults" -o table
```

214. Run a non-disruptive test failover into an isolated VNet

215. Perform a failover to the secondary region and commit

```
az vm list --resource-group "rg-az104-lab26-dr" -o table
```

216. Re-protect and fail back to the primary region

217. Build a Recovery Plan for orchestrated failover

Test it

The replicated item reaches Protected/Healthy with a current RPO, and a test failover creates a working VM in the secondary region.

Note: Full commands (Portal + CLI + PowerShell) are in labs/lab-26-*.md. Clean up when done:

```
az group delete --name rg-az104-lab26 --yes --no-wait
```

Exam Preparation

- First pass: do every lab via the Azure Portal, reading the References in each lab file.
- Second pass: redo the labs using only the Azure CLI until the command verbs are automatic.
- Review the 'Test it' check and the 'What you learned' bullets for any topic you find hard.
- Take the free Microsoft practice assessment for AZ-104.
- Passing score is 700/1000. Book the exam from your Microsoft Learn profile.

Glossary

- **Resource group** — A container that holds related Azure resources sharing a lifecycle.
- **Microsoft Entra ID** — Azure's cloud identity service (authentication) — formerly Azure AD.
- **RBAC** — Role-Based Access Control — grants a principal a role at a scope (authorization).
- **Azure Policy** — Rules that audit or enforce resource configuration for governance.
- **Storage account** — A globally-unique namespace holding Blobs, Files, Queues and Tables.
- **SAS token** — Shared Access Signature — a scoped, time-limited URL for storage access.
- **ARM / Bicep** — Azure Resource Manager templates / the Bicep language for infrastructure as code.
- **VNet** — Virtual Network — your private, isolated network in Azure, divided into subnets.
- **NSG** — Network Security Group — stateful allow/deny rules filtering subnet or NIC traffic.
- **Azure Bastion** — Managed service for browser RDP/SSH to VMs without a public IP.
- **Log Analytics / KQL** — Azure Monitor's log store and its Kusto Query Language.
- **Recovery Services vault** — The container Azure Backup and Site Recovery use to store recovery points.